

OSSP-PRACTICAL-WEEK 3

Task-1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());
    printf("Parent PPID: %d\n", getppid());
    printf("Parent State: Running\n\n");

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    else if (pid == 0)
    {
        printf("Child process started.\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        printf("Child State: Running\n");

        sleep(2);

        printf("Child State: Terminating\n");
        return 0;
    }

    else
    {
        printf("Child process started.\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        printf("Child State: Running\n");

        sleep(2);

        printf("Child State: Terminating\n");
        return 0;
    }

    else
    {
        printf("Parent created child successfully.\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent State: Waiting for child\n");

        wait(NULL);

        printf("Child process completed.\n");
        printf("Parent State: Running\n");
    }

    return 0;
}
```

Output:

```
PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ nano process.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ gcc process.c -o process
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./process
Parent process started.
Parent PID: 450
Parent PPID: 416
Parent State: Running

Parent created child successfully.
Parent PID: 450
Child PID: 451
Parent State: Waiting for child
Child process started.
Child PID: 451
Child PPID: 450
Child State: Running
Child State: Terminating
Child process completed.
Parent State: Running
```

Observation: I learned how the `fork()` system call creates a child process from a parent process. I understood how to display and differentiate the PID and PPID of parent and child processes. I also learned how a parent process waits for the child process to complete using `wait()` and observed the different process states during execution.

Task2: The aim is to observe process state transitions such as Running, Waiting/Sleeping, and Terminated using Linux commands like ps, top, and /proc.

Source code:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process started.\n");
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());

    printf("Process State: Running\n");

    printf("Process State: Sleeping\n");
    sleep(15);

    printf("Process State: Running again\n");
    sleep(5);

    printf("Process State: Terminating\n");

    return 0;
}
```

Output:

```
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./state
Process started.
PID: 578
PPID: 547
Process State: Running
Process State: Sleeping
top

Process State: Running again
Process State: Terminating
```

Using command top():

```
top - 04:20:56 up 1:22, 1 user, load average: 0.03, 0.01, 0.00
Tasks: 35 total, 1 running, 34 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3580.6 total, 2894.4 free, 485.5 used, 280.0 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used, 3095.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
850	sairam0+	20	0	9308	5696	3508	S	0.3	0.2	0:00.01	top
1	root	20	0	21880	13020	9668	S	0.0	0.4	0:01.37	systemd
2	root	20	0	3180	2208	2072	S	0.0	0.1	0:00.04	init-systemd(Ub
7	root	20	0	3180	2040	1940	S	0.0	0.1	0:00.00	init
43	root	19	-1	66772	19516	18400	S	0.0	0.5	0:00.70	systemd-journal
73	root	20	0	25288	6568	5144	S	0.0	0.2	0:00.84	systemd-udev
150	systemd+	20	0	21352	13164	11040	S	0.0	0.4	0:00.29	systemd-resolve
153	systemd+	20	0	91044	7992	7024	S	0.0	0.2	0:00.22	systemd-timesyn
162	root	20	0	4248	2684	2432	S	0.0	0.1	0:00.01	cron
163	message+	20	0	9656	5332	4636	S	0.0	0.1	0:00.16	dbus-daemon
170	root	20	0	17976	8788	7748	S	0.0	0.2	0:00.19	systemd-logind
178	syslog	20	0	222524	5556	4608	S	0.0	0.2	0:00.14	rsyslogd
188	root	20	0	3128	1968	1836	S	0.0	0.1	0:00.02	agetty
192	root	20	0	107052	23256	13804	S	0.0	0.6	0:00.34	unattended-upgr
302	root	20	0	6704	4736	3952	S	0.0	0.1	0:00.08	login
351	sairam0+	20	0	20340	11484	9360	S	0.0	0.3	0:00.23	systemd
352	sairam0+	20	0	21180	3572	1848	S	0.0	0.1	0:00.00	(sd-pam)
376	sairam0+	20	0	6084	5324	3656	S	0.0	0.1	0:00.04	bash
635	root	20	0	3184	1120	980	S	0.0	0.0	0:00.00	SessionLeader
636	root	20	0	3200	1256	1108	S	0.0	0.0	0:00.00	Relay(637)
637	sairam0+	20	0	6084	5416	3696	S	0.0	0.1	0:00.03	bash
703	root	20	0	3184	1120	980	S	0.0	0.0	0:00.00	SessionLeader
704	root	20	0	3200	1128	980	S	0.0	0.0	0:00.00	Relay(708)
708	sairam0+	20	0	6084	5388	3664	S	0.0	0.1	0:00.05	bash
786	root	20	0	3184	1120	980	S	0.0	0.0	0:00.00	SessionLeader
787	root	20	0	3200	1256	1108	S	0.0	0.0	0:00.00	Relay(790)
790	sairam0+	20	0	6084	5392	3668	S	0.0	0.1	0:00.05	bash
806	root	20	0	3184	1120	980	S	0.0	0.0	0:00.00	SessionLeader
807	root	20	0	3200	1128	980	S	0.0	0.0	0:00.00	Relay(813)
813	sairam0+	20	0	6084	5424	3700	S	0.0	0.1	0:00.05	bash
829	sairam0+	20	0	2692	1732	1636	S	0.0	0.0	0:00.00	state
830	root	20	0	3184	1120	980	S	0.0	0.0	0:00.00	SessionLeader
831	root	20	0	3200	1256	1108	S	0.0	0.0	0:00.00	Relay(836)
836	sairam0+	20	0	6084	5388	3664	S	0.0	0.1	0:00.04	bash

Using ps-ef command:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ps -ef | grep week3pt2
sairam0+    1082    1068  50 16:14 pts/3    00:00:00 grep --color=auto week3pt2
```

Observation:

The process was executed and monitored using Linux process monitoring commands. The top command was used to observe the running process, while ps was used to identify the process and its PID. The experiment showed that a process can change between Running and Sleeping/Waiting states during execution. The process finally terminates after completing its execution.

